

Direct Addressing Table

1. Concept

Definition

Direct Addressing is a technique where the key itself is used as the index of an array.

If keys are small integers:

$$\text{Address of key } k = k$$

No hash function is required.

2. Working Principle

We create an array T such that:

$$T[k] = \text{element with key } k$$

If a key does not exist:

$$T[k] = \text{NIL}$$

Access is done using direct array indexing.

3. Example (Small Key Range)

Suppose we store roll numbers:

$$\{2, 5, 8\}$$

Maximum key = 8

So we create:

$$T[0 \dots 8]$$

Visual Representation

0	1	2	3	4	5	6	7	8
		Data			Data			Data

Only indices 2, 5, 8 store values. Others remain empty (NIL).

4. Problem Example (Large Key Range)

Keys:

$\{10, 500000000, 999999999\}$

Maximum key $\approx 10^9$

We must create:

$T[0 \dots 10^9]$

Major Issue

Only 3 elements stored But memory allocated for 10^9 entries!

This causes extreme memory waste.

5. Time Complexity

Performance

Insert = $O(1)$
Search = $O(1)$
Delete = $O(1)$

Reason:

Array indexing is constant time

6. Space Complexity

Let:

- U = Universe size (maximum possible key)
- n = Number of stored elements

Space Required:

$$\boxed{O(U)}$$

Not:

$$O(n)$$

If:

$$U \gg n$$

Then most memory is unused.

7. When Is It Useful?

- Small key range
- U close to n
- Frequency counting
- Counting sort
- Boolean presence arrays

8. Core Trade-Off

Important Insight

Direct Addressing provides:

Perfect $O(1)$ time

But requires:

Huge $O(U)$ space

This trade-off leads to the development of **Hashing**.

Comparison: Direct Addressing vs Hashing

Key Difference

Direct Addressing uses the key itself as the index. Hashing compresses a large key space into a smaller table.

Feature	Direct Addressing	Hashing
Index Computation	$T[k]$	$T[h(k)]$
Space Complexity	$O(U)$	$O(m)$
Handles Large Keys	No	Yes
Memory Efficient	No	Yes
Collision Handling	Not needed	Required

Where:

U = Universe size (max possible key)

m = Hash table size

9. Memory Usage Example (Actual Numbers)

Assume:

- Keys range from 0 to 10^9
- Each table entry stores an 8-byte pointer

- Only $n = 100$ elements are stored

Direct Addressing Memory Requirement

We must create:

$$10^9 + 1 \text{ slots}$$

Each slot = 8 bytes

Total memory:

$$10^9 \times 8 \text{ bytes}$$

$$= 8 \times 10^9 \text{ bytes}$$

$$= 8 \text{ GB}$$

But we are storing only 100 elements!

Memory used for actual data:

$$100 \times 8 = 800 \text{ bytes}$$

Huge waste.

Hashing Memory Requirement

Suppose we choose:

$$m = 200$$

Each slot = 8 bytes

Total memory:

$$200 \times 8 = 1600 \text{ bytes}$$

Much smaller than 8 GB.

Conclusion from Memory Example

Key Insight

Direct Addressing wastes memory when key range is large. Hashing compresses a large universe into a manageable table.

Direct Addressing is practical only when $U \approx n$

Otherwise:

Hashing is the better choice.

**Direct Addressing = Extremely Fast but Memory
Expensive**

Hashing

1. Why Hashing?

Motivation

Direct Addressing wastes memory when key range is very large. Hashing reduces memory while keeping operations nearly constant time.

Instead of storing elements at index k , we compute a smaller index using a **hash function**.

$$h(k) = k \mod m$$

Where:

- k = key
- m = size of hash table

2. Example of Hash Function

Let:

$$m = 10$$

Keys:

10, 22, 31, 4, 15

Compute hash values:

$$h(10) = 0$$

$$h(22) = 2$$

$$h(31) = 1$$

$$h(4) = 4$$

$$h(15) = 5$$

Visual Representation

0	1	2	3	4	5	6	7	8	9
10	31	22		4	15				

Memory usage is now:

$$O(m)$$

instead of $O(U)$.

3. Collision

Definition of Collision

A collision occurs when two different keys map to the same index.

Mathematically:

$$h(k_1) = h(k_2), \quad k_1 \neq k_2$$

Collision Example

Let:

$$m = 10$$

Keys:

$$22, 32$$

$$h(22) = 2$$

$$h(32) = 2$$

Both map to index 2 $\rightarrow$ Collision.

0	1	2	3	4	5	6	7	8	9
		22 32							

4. When Is Collision Guaranteed?

If:

$$n > m$$

Where:

- n = number of keys
- m = table size

Then collision must occur.

Pigeonhole Principle

If more objects than boxes, at least one box contains more than one object.

Therefore:

Collisions are unavoidable in hashing.

5. Properties of a Good Hash Function

Goal

A good hash function distributes keys uniformly across the table to minimize collisions.

1. Uniform Distribution

A good hash function should spread keys evenly over:

$$0, 1, 2, \dots, m - 1$$

If keys are random, then each index should be equally likely:

$$P(h(k) = i) = \frac{1}{m}$$

Why important?

- Reduces clustering
- Reduces collisions
- Ensures average $O(1)$ performance

2. Deterministic

For the same key k :

$h(k)$ must always produce the same value

Otherwise searching becomes impossible.

3. Fast to Compute

Hash function must be:

$$O(1)$$

It should use simple arithmetic operations.

4. Uses All Key Information

A good hash function should depend on all bits of the key.

Small change in key $\rightarrow$ large change in hash value.

This property is called:

Avalanche Effect

Example:

$$h(12345) \neq h(12346)$$

5. Minimizes Patterns

If input keys follow a pattern, hash output should not preserve that pattern.

Example:

Sequential keys should not map to sequential indices.

6. Examples of Poor Hash Functions

Poor Hash Functions Cause Heavy Collisions

Bad hash functions destroy performance.

Example 1: Constant Hash Function

$$h(k) = 1$$

All keys map to index 1.

Result:

- Maximum collisions
- Time complexity becomes $O(n)$
- Equivalent to linked list

Worst possible hash function.

Example 2: Using Only Last Digit

$$h(k) = k \bmod 10$$

If table size $m = 10$ and keys are:

10, 20, 30, 40, 50

Then:

$$h(10) = 0$$

$$h(20) = 0$$

$$h(30) = 0$$

All collide at index 0.

Poor distribution.

Example 3: Ignoring Higher Bits

Suppose keys are large numbers but we use:

$$h(k) = k \bmod 8$$

If $m = 8$ and keys differ only in higher bits:

256, 512, 1024

All are multiples of 256:

$$h(256) = 0$$

$$h(512) = 0$$

$$h(1024) = 0$$

Severe clustering.

Example 4: Sequential Mapping

$$h(k) = k$$

If table size is small and keys are large sequential numbers, collisions occur frequently.

Better:

$$h(k) = k \bmod m$$

Comparison: Good vs Poor Hash Function

Property	Good	Poor
Uniform distribution	✓	✗
Uses full key	✓	✗
Fast computation	✓	✓
Avoids clustering	✓	✗
Maintains $O(1)$ avg	✓	✗

A Hash Table is Only as Good as its Hash Function

Hashing = Memory Efficient + Near Constant Time

Chaining (Separate Chaining)

1. Concept

Definition

Chaining is a collision resolution technique where each hash table index stores a linked list of elements that hash to the same value.

Instead of storing only one element per index, we store a **list of elements**.

2. Example

Let:

$$m = 5$$

Hash function:

$$h(k) = k \mod 5$$

Keys:

$$12, 7, 22, 17, 5$$

Compute hash values:

$$h(12) = 2$$

$$h(7) = 2$$

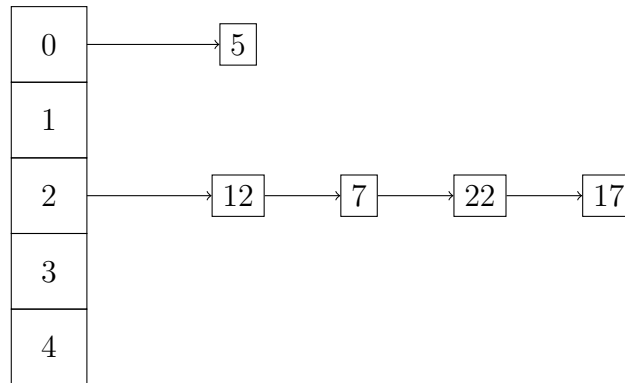
$$h(22) = 2$$

$$h(17) = 2$$

$$h(5) = 0$$

Many keys map to index 2 $\rightarrow$ Collision handled using linked list.

3. Visual Representation



Index 2 stores a linked list of 4 elements.

4. Time Complexity

Let:

- n = number of elements
- m = table size

$$\alpha = \frac{n}{m}$$

where α is called the **Load Factor**.

Insertion

Insert at head of list:

$$O(1)$$

Search / Delete

Worst case (all keys in one bucket):

$$O(n)$$

Average case:

$$O(1 + \alpha)$$

Why?

Because average chain length is:

$$\alpha = \frac{n}{m}$$

5. Important Observations

Key Insight

If α is small (for example $\alpha < 1$), then chaining gives nearly constant time performance.

- Works even if $\alpha > 1$
 - No clustering problem
 - Easy to implement
 - Requires extra memory for linked lists
-

6. Worst-Case Scenario in Chaining

Worst Case: All Keys in One Bucket

If all keys hash to the same index, the hash table degenerates into a linked list.

Suppose:

$$h(k_1) = h(k_2) = \dots = h(k_n)$$

Then the bucket contains:

n elements

Time complexity becomes:

$$\text{Search} = O(n)$$

$$\text{Insert} = O(1) \quad (\text{if inserted at head})$$

$$\text{Delete} = O(n)$$

This is equivalent to having no hashing at all.

Visual Intuition:

Instead of:

m small linked lists

We get:

1 long linked list

Which destroys the benefit of hashing.

Dynamic Resizing (Rehashing)

Why Resize?

As the number of elements increases, load factor increases, leading to longer chains and slower searches.

Load factor:

$$\alpha = \frac{n}{m}$$

When:

$$\alpha > 1 \quad (\text{or chosen threshold})$$

Performance degrades.

Solution: Rehashing

Steps:

1. Create a new larger table (usually $2\times$ size)
2. Choose a new prime table size
3. Recompute hash values for all elements
4. Insert them into new table

Time Complexity of Rehashing:

$$O(n)$$

But since resizing happens occasionally, the amortized cost per insertion remains:

$$O(1)$$

Practical Thresholds:

- Resize when $\alpha > 1$ (common in chaining)
- Some implementations resize at $\alpha > 0.75$

Chaining Works Well — If Load Factor Is Controlled

Chaining = Flexible and Robust Collision Handling

Open Addressing

1. Concept

Definition

Open Addressing is a collision resolution technique in which all elements are stored directly inside the hash table itself.

Unlike chaining, no linked lists are used.

When a collision occurs, we search for another empty slot using a probing sequence.

2. General Probing Formula

$$h(k, i) = \text{Probe Function}$$

Where:

- k = key
- i = probe number ($i = 0, 1, 2, \dots$)
- m = table size

First attempt:

$$h(k, 0)$$

If occupied $\rightarrow$ try:

$$h(k, 1), h(k, 2), \dots$$

until an empty slot is found.

3. Example

Let:

$$m = 7$$

$$h(k) = k \mod 7$$

Insert keys:

10, 17, 24

$$h(10) = 3$$

$$h(17) = 3$$

$$h(24) = 3$$

All collide at index 3.

Open addressing finds new empty slots.

Visual Representation

0	1	2	3	4	5	6
			10	17	24	

Elements are placed in nearby empty slots.

4. Types of Open Addressing

- Linear Probing
- Quadratic Probing
- Double Hashing

Each method defines a different probe function.

5. Load Factor Condition

$$\alpha = \frac{n}{m}$$

Where:

- n = number of elements
- m = table size

Important Condition

For Open Addressing:

$$\alpha < 1$$

Table must have at least one empty slot.

If $\alpha \rightarrow 1$, performance degrades rapidly.

6. Advantages and Disadvantages

Advantages

- No extra memory for linked lists
- Better cache performance
- Simple array implementation

Disadvantages

- Clustering problem
- Performance sensitive to load factor
- Deletion is more complicated

Open Addressing = Compact Memory but Load Sensitive

Linear Probing

1. Concept

Definition

Linear Probing is an open addressing collision resolution technique where we sequentially search for the next empty slot.

$$h'(k) = k$$

$$h(k, i) = (h'(k) + i) \bmod m$$

Where:

- k = key
- i = probe number $(0, 1, 2, \dots)$
- m = table size

2. How Linear Probing Works

Step 1: Compute initial index

$$h(k, 0)$$

Step 2: If occupied, try

$$h(k, 1), h(k, 2), \dots$$

Move sequentially until an empty slot is found.

3. Simulation Example

Let:

$$m = 11$$

Insert keys:

10, 22, 31, 4, 15, 28, 17, 88, 89

Initial hash:

$$h(k, 0) = k \bmod 11$$

Step-by-Step Insertion (m = 11)

Hash function:

$$h'(k) = k$$

$$h(k, i) = (h'(k) + i) \bmod 11$$

Initial probe:

$$h(k, 0) = k \bmod 11$$

1) Insert 10

$$h(10, 0) = 10 \bmod 11 = 10$$

Index 10 is empty $\rightarrow$ Insert.

Probes = 1

2) Insert 22

$$h(22, 0) = 22 \bmod 11 = 0$$

Index 0 is empty $\rightarrow$ Insert.

Probes = 1

3) Insert 31

$$h(31, 0) = 31 \bmod 11 = 9$$

Index 9 is empty $\rightarrow$ Insert.

Probes = 1

4) Insert 4

$$h(4, 0) = 4 \bmod 11 = 4$$

Index 4 is empty $\rightarrow$ Insert.

Probes = 1

5) Insert 15

$$h(15, 0) = 15 \bmod 11 = 4$$

Index 4 is occupied $\rightarrow$ Collision.

Try next probe:

$$h(15, 1) = (15 + 1) \bmod 11 = 5$$

Index 5 is empty $\rightarrow$ Insert.

Probes = 2

6) Insert 28

$$h(28, 0) = 28 \bmod 11 = 6$$

Index 6 is empty $\rightarrow$ Insert.

Probes = 1

7) Insert 17

$$h(17, 0) = 17 \bmod 11 = 6$$

Index 6 occupied $\rightarrow$ Collision.

$$h(17, 1) = (17 + 1) \bmod 11 = 7$$

Index 7 empty $\rightarrow$ Insert.

Probes = 2

8) Insert 88

$$h(88, 0) = 88 \bmod 11 = 0$$

Index 0 occupied $\rightarrow$ Collision.

$$h(88, 1) = (88 + 1) \bmod 11 = 1$$

Index 1 empty $\rightarrow$ Insert.

Probes = 2

9) Insert 89

$$h(89, 0) = 89 \bmod 11 = 1$$

Index 1 occupied $\rightarrow$ Collision.

$$h(89, 1) = (89 + 1) \bmod 11 = 2$$

Index 2 empty $\rightarrow$ Insert.

Probes = 2

Insertion Summary Table

Key	Final Index	Probes
10	10	1
22	0	1
31	9	1
4	4	1
15	5	2
28	6	1
17	7	2
88	1	2
89	2	2

Total Probes = 13

$\alpha = \frac{9}{11} \approx 0.82$

4. Final Hash Table Visualization

0	1	2	3	4	5	6	7	8	9	10
22	88	89		4	15	28	17		31	10

5. Primary Clustering

Primary Clustering

Sequential probing (+1 each time) causes consecutive occupied cells to form clusters. Future insertions near that cluster require more probes.

Example cluster: indices 0–2.

6. Time Complexity

Let:

$$\alpha = \frac{n}{m}$$

Under uniform hashing assumption:

$$\text{Average successful search} \approx \frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right)$$

As:

$$\alpha \rightarrow 1$$

Performance degrades rapidly.

7. Advantages and Disadvantages

Advantages

- Simple implementation
- No extra memory
- Cache-friendly

Disadvantages

- Primary clustering
- Performance sensitive to load factor
- Deletion requires special handling (lazy deletion)

Linear Probing = Simple but Clustering-Prone

Quadratic Probing

1. Concept

Definition

Quadratic Probing is an open addressing technique in which the probe sequence grows quadratically instead of linearly.

Instead of adding +1 each time, we use:

$$h'(k) = k$$

$$h(k, i) = (h'(k) + C_1i + C_2i^2) \bmod m$$

For this example:

$$C_1 = 1, \quad C_2 = 3$$

$$h(k, i) = (k + i + 3i^2) \bmod 11$$

Where:

- k = key
- i = probe number
- $m = 11$

2. Why Quadratic Probing?

Linear probing causes:

$$+1, +1, +1, +1, \dots$$

This creates large continuous clusters (Primary Clustering).

Quadratic probing spreads probes non-linearly:

$$+1, +4, +9, +16, \dots$$

This reduces clustering.

3. Simulation Example

$$m = 11$$

Insert keys:

$$10, 22, 31, 4, 15, 28, 17, 88, 89$$

Initial probe:

$$h(k, 0) = k \bmod 11$$

4. Step-by-Step Insertion

1) Insert 10

$$h(10, 0) = 10$$

Insert at 10 (1 probe)

2) Insert 22

$$h(22, 0) = 0$$

Insert at 0 (1 probe)

3) Insert 31

$$h(31, 0) = 9$$

Insert at 9 (1 probe)

4) Insert 4

$$h(4, 0) = 4$$

Insert at 4 (1 probe)

5) Insert 15

$$h(15, 0) = 4 \quad (\text{collision})$$

$$h(15, 1) = (15 + 1 + 3) = 19 \bmod 11 = 8$$

Insert at 8 (2 probes)

6) Insert 28

$$h(28, 0) = 6$$

Insert at 6 (1 probe)

7) Insert 17

$$h(17, 0) = 6 \quad (\text{collision})$$

$$h(17, 1) = 10 \quad (\text{collision})$$

$$h(17, 2) = 9 \quad (\text{collision})$$

$$h(17, 3) = 3$$

Insert at 3 (4 probes)

8) Insert 88

Since:

$$88 \bmod 11 = 0$$

We simplify:

$$h(88, i) = (i + 3i^2) \bmod 11$$

Compute sequentially:

$$i = 0 \rightarrow 0(\text{collision})$$

$$i = 1 \rightarrow 4(\text{collision})$$

$$\begin{aligned}
i = 2 &\rightarrow 3(\text{collision}) \\
i = 3 &\rightarrow 8(\text{collision}) \\
i = 4 &\rightarrow 8(\text{collision}) \\
i = 5 &\rightarrow 3(\text{collision}) \\
i = 6 &\rightarrow 4(\text{collision}) \\
i = 7 &\rightarrow 0(\text{collision}) \\
i = 8 &\rightarrow 2
\end{aligned}$$

Index 2 is empty.

Insert at 2.

Probes = 9

9) Insert 89

$$89 \bmod 11 = 1$$

$$h(89, 0) = 1$$

Index 1 empty.

Insert (1 probe)

5. Final Insertion Table

Key	Final Index	Probes
10	10	1
22	0	1
31	9	1
4	4	1
15	8	2
28	6	1
17	3	4
88	2	9
89	1	1

Total Probes = 21

$\alpha = \frac{9}{11} \approx 0.82$

6. Final Hash Table

0	1	2	3	4	5	6	7	8	9	10
22	89	88	17	4		28		15	31	10

7. Clustering Behavior

Observation

Quadratic probing reduces primary clustering, but keys with same initial hash follow identical probe sequences (Secondary Clustering).

8. Important Theoretical Points

- Quadratic probing may not visit all table slots.
- Proper choice of m , C_1 , C_2 is important.
- Often works well when:

$$\alpha \leq 0.5$$

- Does not completely eliminate clustering.

**Quadratic Probing = Less Clustering but More
Complex Probing**

Double Hashing

1. Concept

Definition

Double Hashing resolves collisions using two independent hash functions. The second hash function determines the probe step size.

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

Where:

- $h_1(k)$ = primary hash
- $h_2(k)$ = step size
- m = table size

$$h_1(k) = k \bmod m$$

$$h_2(k) = 1 + (k \bmod (m - 1))$$

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

Where:

- k = key
- i = probe number ($i = 0, 1, 2, \dots$)
- m = table size (preferably prime)

For this example:

$$m = 11$$

$$h_1(k) = k \bmod 11$$

$$h_2(k) = 1 + (k \bmod 10)$$

$$h(k, i) = (h_1(k) + ih_2(k)) \bmod 11$$

2. Why This Design?

- $h_2(k) \neq 0$ (guaranteed by $+1$)
- Step size depends on key
- Eliminates primary clustering
- Eliminates secondary clustering
- Ensures full coverage if m is prime

3. Complete Coverage Proof

Theorem

If m is prime and $\gcd(h_2(k), m) = 1$, then the probe sequence visits all m slots.

Assume:

$$h_1(k) + ih_2(k) \equiv h_1(k) + jh_2(k) \pmod{m}$$

Then:

$$(i - j)h_2(k) \equiv 0 \pmod{m}$$

Since m is prime and $h_2(k)$ is relatively prime to m :

$$i = j$$

Hence all probes are distinct.

Complete Coverage Proven

4. Simulation Example

Insert:

10, 22, 31, 4, 15, 28, 17, 88, 89

$$m = 11$$

5. Step-by-Step Insertion

1) Insert 10

$$h_1 = 10, \quad h_2 = 1$$

Index 10 empty $\rightarrow$ Insert.

Probes = 1

2) Insert 22

$$h_1 = 0, \quad h_2 = 3$$

Index 0 empty $\rightarrow$ Insert.

Probes = 1

3) Insert 31

$$h_1 = 9, \quad h_2 = 2$$

Index 9 empty $\rightarrow$ Insert.

Probes = 1

4) Insert 4

$$h_1 = 4, \quad h_2 = 5$$

Index 4 empty $\rightarrow$ Insert.

Probes = 1

5) Insert 15

$$h_1 = 4, \quad h_2 = 6$$

Index 4 occupied

$$(4 + 6) \bmod 11 = 10 \quad (\text{occupied})$$

$$(4 + 12) \bmod 11 = 5$$

Insert at 5.

Probes = 3

6) Insert 28

$$h_1 = 6, \quad h_2 = 9$$

Index 6 empty $\rightarrow$ Insert.

Probes = 1

7) Insert 17

$$h_1 = 6, \quad h_2 = 8$$

Index 6 occupied

$$(6 + 8) \bmod 11 = 3$$

Insert at 3.

Probes = 2

8) Insert 88

$$h_1 = 0, \quad h_2 = 9$$

Index 0 occupied

$$(0 + 9) \bmod 11 = 9 \quad (\text{occupied})$$

$$(0 + 18) \bmod 11 = 7$$

Insert at 7.

Probes = 3

9) Insert 89

$$h_1 = 1, \quad h_2 = 10$$

Index 1 empty $\rightarrow$ Insert.

Probes = 1

6. Final Insertion Table

Key	Final Index	Probes
10	10	1
22	0	1
31	9	1
4	4	1
15	5	3
28	6	1
17	3	2
88	7	3
89	1	1

$$\boxed{\text{Total Probes} = 14}$$

$$\boxed{\alpha = \frac{9}{11} \approx 0.82}$$

7. Final Hash Table

0	1	2	3	4	5	6	7	8	9	10
22	89		17	4	15	28	88		31	10

8. Time Complexity

$$\alpha = \frac{n}{m}$$

Expected unsuccessful search:

$$E[U] = \frac{1}{1 - \alpha}$$

Expected successful search:

$$E[S] = \frac{1}{\alpha} \ln \frac{1}{1 - \alpha}$$

Double hashing gives the best distribution among open addressing methods.

**Double Hashing = Best Practical Open Addressing
Technique**

Load Factor

1. Definition

Formal Definition

Load Factor measures how full a hash table is.

$$\alpha = \frac{n}{m}$$

Where:

- n = number of stored elements
- m = size of hash table
- α = load factor

Interpretation:

- $\alpha = 0 \rightarrow$ table empty
- $\alpha = 0.5 \rightarrow$ half full
- $\alpha = 1 \rightarrow$ completely full
- $\alpha > 1 \rightarrow$ possible only in chaining

2. Intuition Behind Load Factor

Load factor tells us the average number of elements per slot.

Average elements per slot = α

Example:

If:

$$n = 9, \quad m = 11$$

$$\alpha = \frac{9}{11} \approx 0.82$$

Meaning:

On average, each slot is 82% full.

3. Effect on Collisions

Key Observation

As α increases, the probability of collision increases.

When:

$$\alpha \rightarrow 1$$

Almost every insertion causes a collision.

4. Load Factor in Different Techniques

A. Chaining

In chaining:

$$\alpha = \frac{n}{m}$$

Average search time:

$$O(1 + \alpha)$$

- α can be greater than 1
- Table still works
- Performance degrades linearly

B. Open Addressing

In open addressing:

$$\boxed{\alpha < 1}$$

Table must always have at least one empty slot.

As:

$$\alpha \rightarrow 1$$

Performance degrades dramatically.

C. Linear Probing

Expected unsuccessful search:

$$E[U] \approx \frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right)$$

When:

$$\alpha = 0.9$$

Performance becomes very poor.

D. Double Hashing

Expected unsuccessful search:

$$E[U] = \frac{1}{1 - \alpha}$$

Performs better than linear probing at high load factors.

5. Visual Understanding

$$\alpha = 0.8$$



80% of the table is filled.

6. When to Resize (Rehashing)

When load factor exceeds a threshold:

$$\alpha > 0.7 \text{ (commonly used threshold)}$$

We:

- Create a new larger table
- Recompute hash values
- Reinsert all elements

This process is called:

Rehashing

7. Practical Recommendations

- For chaining: keep $\alpha \leq 1$
- For open addressing: keep $\alpha \leq 0.7$
- Always use prime table size
- Resize dynamically

Load Factor Controls Performance of Hash Tables

Hashing

Collision Resolution by Chaining

1 Collision Resolution by Chaining

When multiple keys hash to the same slot, a **collision** occurs.

In **chaining**, we resolve collisions by storing all elements that hash to the same slot in a linked list.

Important Concept

Each slot j of the hash table contains:

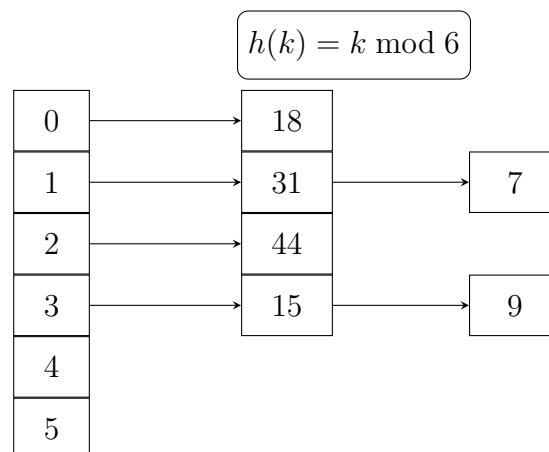
$$T[j] = \text{pointer to head of linked list}$$

If no elements hash to slot j , then:

$$T[j] = \text{NIL}$$

Thus, the hash table is an array of linked lists.

2 Hashing with Chaining



3 Dictionary Operations with Chaining

Let $h(k)$ be the hash function.

1. Insertion

Algorithm:

- Compute $j = h(x.key)$
- Insert x at the **head** of list $T[j]$

Worst-case time: $O(1)$

Key Idea

Insertion is fast because we assume the key is not already present. If duplicate checking is required, we must search first (extra cost).

2. Search

Algorithm:

- Compute $j = h(k)$
- Search linked list $T[j]$

Worst-case time: $\Theta(n)$

(Occurs when all keys hash to the same slot.)

3. Deletion

If lists are **doubly linked**:

Deletion time = $O(1)$

If lists are **singly linked**:

- Must search for predecessor
- Deletion becomes $\Theta(n)$

Important Concept

If deletion is required, use **doubly linked lists**.

4 Load Factor

Definition 1 (Load Factor). If a hash table has:

- m slots
- n stored elements

Then the load factor is:

$$\alpha = \frac{n}{m}$$

α = average number of elements per chain

- $\alpha < 1 \rightarrow$ Sparse table
- $\alpha = 1 \rightarrow$ Balanced
- $\alpha > 1 \rightarrow$ Heavy chaining

5 Worst-Case Behavior

Worst case occurs when:

All n keys hash to the same slot

Search time = $\Theta(n)$

This is no better than using a single linked list.

Important Concept

Hash tables are not designed for worst-case performance. They are optimized for **average-case performance**.

6 Simple Uniform Hashing Assumption

We assume:

Each key is equally likely to hash to any of the m slots, independently.

Simple Uniform Hashing

Let n_j denote the length of chain j (i.e., the number of keys that hash to slot j).

We compute its expected value.

For each key k_i , define the indicator variable:

$$Y_{ij} = \begin{cases} 1 & \text{if } h(k_i) = j \\ 0 & \text{otherwise} \end{cases}$$

Then the total number of elements in chain j is:

$$n_j = \sum_{i=1}^n Y_{ij}$$

Taking expectation and applying linearity:

$$E[n_j] = E \left[\sum_{i=1}^n Y_{ij} \right] = \sum_{i=1}^n E[Y_{ij}]$$

Since each key is equally likely to hash to any of the m slots,

$$\Pr(h(k_i) = j) = \frac{1}{m}$$

and because Y_{ij} is an indicator variable,

$$E[Y_{ij}] = \frac{1}{m}$$

Therefore,

$$E[n_j] = \sum_{i=1}^n \frac{1}{m} = \frac{n}{m} = \alpha$$

$$\boxed{E[n_j] = \alpha}$$

where n_j is the length of chain j .

7 Unsuccessful Search Analysis

Theorem 1. *In chaining with simple uniform hashing, an unsuccessful search takes*

$$\Theta(1 + \alpha)$$

average-case time.

Proof Idea:

- Expected chain length = α
- Examine all elements in that chain
- Add $O(1)$ time for computing $h(k)$

$$\text{Expected time} = \alpha + O(1) = \Theta(1 + \alpha)$$

8 Successful Search Analysis

Theorem 2. *In a hash table with chaining under simple uniform hashing, a successful search takes*

$$\Theta(1 + \alpha)$$

average-case time.

Proof

Assume:

- n elements stored.
- Inserted in order $x_1, x_2, \dots, x_n$.
- Insertions occur at the front.
- Search target equally likely among n elements.

Step 1: Elements Examined

$$\text{Elements examined} = 1 + \sum_{j=i+1}^n X_{ij}$$

Step 2: Indicator Variables

For keys k_i and k_j ($j > i$), define the indicator random variable

$$X_{ij} = \begin{cases} 1 & \text{if } h(k_i) = h(k_j) \\ 0 & \text{otherwise} \end{cases}$$

Thus, X_{ij} indicates whether the two keys collide.

Since X_{ij} is an indicator random variable, we use the fundamental property:

$$E[X] = \Pr(X = 1)$$

Therefore,

$$E[X_{ij}] = \Pr(h(k_i) = h(k_j))$$

Under the assumption of **simple uniform hashing**:

- Each key is equally likely to hash into any of the m slots.
- Hash values are independent.

If k_i hashes to some slot, the probability that k_j hashes to the same slot is:

$$\Pr(h(k_i) = h(k_j)) = \frac{1}{m}$$

Hence,

$$\boxed{E[X_{ij}] = \frac{1}{m}}$$

Step 3: Expected Value

$$\begin{aligned} E &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij} \right) \\ &= \frac{1}{n} \sum_{i=1}^n \left(1 + \frac{n-i}{m} \right) \end{aligned}$$

Step 4: Simplification

$$= \frac{1}{n} \left(n + \frac{1}{m} \cdot \frac{n(n-1)}{2} \right) = 1 + \frac{n-1}{2m}$$

Step 5: Express Using Load Factor

$$\alpha = \frac{n}{m}$$

$$E = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

Including hash computation:

$$\Theta \left(2 + \frac{\alpha}{2} - \frac{\alpha}{2n} \right) = \Theta(1 + \alpha)$$

□

9 Final Interpretation

If:

$$n = O(m)$$

then

$$\alpha = O(1)$$

$$\boxed{\text{Search runs in } O(1) \text{ average time.}}$$

Insertion: $O(1)$ worst-case

Deletion: $O(1)$ worst-case (doubly linked)

$O(1)$ average time for all dictionary operations

10 Quick Exam Summary

- Collision resolution method: **Chaining**
- Load factor: $\alpha = n/m$
- Unsuccessful search: $\Theta(1 + \alpha)$
- Successful search: $\Theta(1 + \alpha)$
- Worst case: $\Theta(n)$
- Works well when $\alpha = O(1)$

Chaining gives constant average time — if the hash function distributes keys uniformly.

Analysis of Open-Address Hashing

Load Factor and Uniform Hashing

Let:

$$\alpha = \frac{n}{m}$$

where:

- n = number of stored elements
- m = number of slots

In open addressing:

$$n \leq m \quad \Rightarrow \quad \alpha \leq 1$$

We assume **uniform hashing**:

Each probe sequence

$$\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$$

is equally likely to be any permutation of $\{0, 1, \dots, m-1\}$.

This is an idealized assumption used for analysis.

Unsuccessful Search

Theorem 1 (11.6). *Given an open-address hash table with load factor $\alpha < 1$, the expected number of probes in an unsuccessful search is at most*

$$\frac{1}{1 - \alpha}.$$

Proof

Let X be the number of probes in an unsuccessful search.

An unsuccessful search continues probing until it encounters an empty slot. Every probe except the last must hit an occupied slot.

Define event A_i :

A_i = event that the i th probe occurs and hits an occupied slot

Thus, for at least i probes to occur, the first $i - 1$ probes must all hit occupied slots.

$$\{X \geq i\} = A_1 \cap A_2 \cap \cdots \cap A_{i-1}$$

By the rule of conditional probability:

$$\Pr(X \geq i) = \Pr(A_1) \Pr(A_2|A_1) \cdots \Pr(A_{i-1}|A_1 \cdots A_{i-2})$$

Now we compute these probabilities.

Since there are n occupied slots among m total slots:

$$\Pr(A_1) = \frac{n}{m}$$

For $j > 1$, given that the first $j - 1$ probes hit distinct occupied slots, there remain:

$$n - (j - 1) \text{ occupied slots}$$

and

$$m - (j - 1) \text{ unexamined slots.}$$

Under uniform hashing, every remaining slot is equally likely to be probed next. Therefore,

$$\Pr(A_j|A_1 \cdots A_{j-1}) = \frac{n - (j - 1)}{m - (j - 1)}.$$

Since $n < m$, we have for all j :

$$\frac{n - (j - 1)}{m - (j - 1)} \leq \frac{n}{m} = \alpha.$$

Therefore,

$$\Pr(X \geq i) \leq \alpha^{i-1}.$$

Now we compute the expectation.

A useful identity for nonnegative integer random variables is:

$$E[X] = \sum_{i=1}^{\infty} \Pr(X \geq i).$$

Using the bound above:

$$E[X] \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \sum_{i=0}^{\infty} \alpha^i.$$

This is a geometric series, and since $\alpha < 1$:

$$\sum_{i=0}^{\infty} \alpha^i = \frac{1}{1-\alpha}.$$

Hence,

$$\boxed{E[X] \leq \frac{1}{1-\alpha}}$$

□

Intuition Behind the Result

An unsuccessful search behaves like repeatedly “trying” slots until finding an empty one.

- We always perform the first probe.
- With probability approximately α , the first slot is occupied, so we probe again.
- With probability approximately α^2 , the first two slots are occupied.
- With probability approximately α^3 , the first three are occupied.

Thus, the expected number of probes behaves like:

$$1 + \alpha + \alpha^2 + \alpha^3 + \cdots = \frac{1}{1-\alpha}.$$

Numerical Examples

If the table is half full:

$$\alpha = 0.5 \quad \Rightarrow \quad E[X] \leq \frac{1}{1 - 0.5} = 2.$$

If the table is 90% full:

$$\alpha = 0.9 \quad \Rightarrow \quad E[X] \leq \frac{1}{1 - 0.9} = 10.$$

This shows that performance deteriorates rapidly as α approaches 1.

Insertion

Corollary 1 (11.7). *Inserting an element requires at most*

$$\frac{1}{1 - \alpha}$$

probes on average.

Proof

Insertion consists of:

- Performing an unsuccessful search to find an empty slot.
- Placing the element in the first empty slot found.

Since the expected cost of an unsuccessful search is at most $\frac{1}{1-\alpha}$, the same bound applies to insertion.

$$E[\text{probes for insertion}] \leq \frac{1}{1 - \alpha}$$

□

Successful Search

Theorem 2 (11.8). *Given an open-address table with load factor $\alpha < 1$, the expected number of probes in a successful search is at most*

$$\frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right).$$

Proof

A successful search for a key k follows exactly the same probe sequence that was used when k was inserted. This is because open addressing stores the key in the first empty slot encountered during insertion, and searching follows the identical probe sequence until it finds that key.

Suppose k was the $(i + 1)$ st key inserted into the table. At that time, exactly i keys were already present, so the load factor during its insertion was:

$$\frac{i}{m}.$$

By Theorem 11.6 (unsuccessful search bound), the expected number of probes needed to insert this key was at most:

$$\frac{1}{1 - i/m} = \frac{m}{m - i}.$$

Since a successful search for k retraces the same probes, the expected number of probes in searching for this key is also at most:

$$\frac{m}{m - i}.$$

Now we average over all n stored keys, assuming each key is equally likely to be searched for.

$$E[X] = \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m - i}.$$

Factor out m :

$$= \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m - i}.$$

Since $\alpha = n/m$, we have:

$$\frac{m}{n} = \frac{1}{\alpha}.$$

Thus,

$$E[X] = \frac{1}{\alpha} \sum_{i=0}^{n-1} \frac{1}{m - i}.$$

Reindex the summation by letting:

$$k = m - i.$$

As i goes from 0 to $n - 1$, k goes from m down to $m - n + 1$.

Thus:

$$E[X] = \frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k}.$$

The sum

$$\sum_{k=m-n+1}^m \frac{1}{k}$$

is part of the harmonic series.

To bound it, we use the fact that for a decreasing function $1/x$:

$$\sum_{k=a}^b \frac{1}{k} \leq \int_{a-1}^b \frac{1}{x} dx.$$

Applying this inequality:

$$\sum_{k=m-n+1}^m \frac{1}{k} \leq \int_{m-n}^m \frac{1}{x} dx.$$

Compute the integral:

$$= \ln m - \ln(m - n) = \ln \left(\frac{m}{m - n} \right).$$

Since:

$$\alpha = \frac{n}{m} \quad \Rightarrow \quad m - n = m(1 - \alpha),$$

we obtain:

$$\frac{m}{m - n} = \frac{m}{m(1 - \alpha)} = \frac{1}{1 - \alpha}.$$

Therefore,

$$E[X] \leq \frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right).$$

□

Intuition Behind the Result

When the table is nearly empty, most keys were inserted when few slots were occupied, so they required very few probes.

When the table becomes more full, later insertions require more probes.

A successful search averages over all these insertions, which leads to a harmonic-type growth. This is why the logarithm appears in the bound.

The growth is much slower than the unsuccessful case. Even when α is large, the performance remains moderate.

Numerical Examples

If $\alpha = 0.5$:

$$E[X] \leq \frac{1}{0.5} \ln \left(\frac{1}{0.5} \right) = 2 \ln 2 \approx 1.387.$$

If $\alpha = 0.9$:

$$E[X] \leq \frac{1}{0.9} \ln \left(\frac{1}{0.1} \right) \approx 2.559.$$

Notice that even at 90% load, successful search is still relatively small, while unsuccessful search would require about 10 probes.

Final Summary

Operation	Expected Probes
Unsuccessful Search	$\frac{1}{1 - \alpha}$
Insertion	$\frac{1}{1 - \alpha}$
Successful Search	$\frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right)$

Open addressing performs well only when α is kept significantly below 1.